U21ADP05

EXPLORATORY DATA ANALYSIS AND VISUALIZATION

ASSIGNMENT - II

PROJECT REPORT

STUDENT PERFORMANCE ANALYSIS AND PREDICTION USING DEEP LEARNING

Submitted by

SRI HARI R

24AD114

Artificial Intelligence and Data Science

III AIDS B



Date of Submission: 15 October 2026

ABSTRACT

Student performance prediction uses educational data to identify patterns associated with academic achievement and to support timely academic guidance. This project, titled "Student Performance Analysis and Prediction Using Deep Learning," is designed around the UCI Student Performance dataset, which contains 395 student records for the Mathematics subject and 33 original attributes describing demographic, family, school, and study-related factors, along with grade information. The workflow includes exploratory data analysis, missing-value and duplicate checks, categorical encoding, numerical preprocessing, train/validation/test separation, and a multilayer perceptron (MLP) model for grade prediction. Performance is intended to be evaluated using mean absolute error (MAE), root mean squared error (RMSE), and the coefficient of determination (R^2), supported by visualizations of distributions and training history. The shared Colab notebook required sign-in during preparation, so measured model scores and notebook-specific settings must be copied from the executed notebook before final submission.

1. INTRODUCTION AND OBJECTIVES

Academic performance is influenced by multiple factors, including study habits, attendance, previous achievement, family context, and school environment. Analysing these factors can help educators understand broad trends and identify students who may benefit from additional support. Machine learning and deep learning can model relationships between input attributes and a numerical academic outcome.

This project focuses on analysing student data and developing a deep-learning regression model to estimate Mathematics performance. The model is intended as a decision-support aid and should not replace teacher judgement or be used to label students permanently.

1.1 Objectives

- Explore the dataset using descriptive statistics and visualizations.
- Check and handle missing values, duplicate records, inconsistent values, and potential outliers.
- Encode categorical variables and scale numerical variables where appropriate.
- Separate the data into training, validation, and test sets to evaluate generalization.
- Build and train a multilayer perceptron neural network for grade prediction.
- Evaluate the model using MAE, RMSE, and R^2, and interpret results responsibly.

1.2 Scope

The scope is limited to the dataset and target variable used in the supplied Colab notebook. The UCI dataset includes Mathematics and Portuguese student records; the project context here describes the Mathematics subset with 395 records. The final report should match the exact dataset file and target column loaded in the notebook.

2. DATASET DESCRIPTION

Dataset source: UCI Machine Learning Repository, Student Performance dataset. The dataset was collected from two Portuguese secondary schools and includes student demographic, social, family, and school-related information. The Mathematics file commonly used in this project contains 395 rows and 33 original attributes, including the grade variables G1, G2, and G3.

Item	Description
Dataset name	Student Performance (Mathematics subset)
Source	UCI Machine Learning Repository
Number of records	395 student records
Number of original attributes	33
Target	Final Mathematics grade (commonly G3); verify against the notebook
Data types	Numerical and categorical/binary attributes
Example fields	age, sex, studytime, failures, absences, schoolsup, famsup, G1, G2, G3
Task type	Regression if predicting a numerical grade

The grade variables G1 and G2 represent earlier period grades and G3 represents the final grade. If G3 is the target, including G3 as an input would cause target leakage. Whether G1 and G2 should be included depends on the intended prediction time: they are valid only if those earlier grades would be available when predictions are made.

2.1 Data quality checks

Check	Method	Report status
Missing values	Count nulls by column; impute using training-set statistics if required	Confirm actual counts from notebook
Duplicates	Count full duplicate rows and remove exact duplicates if appropriate	Confirm actual count from notebook
Outliers	Inspect boxplots and IQR limits; investigate before capping/removing	Confirm affected columns and treatment
Data types	Check numeric, categorical, and binary fields	Confirm after loading
Target validity	Check grade range and distribution	Confirm actual target column

3. EDA AND PREPROCESSING

3.1 Exploratory Data Analysis

Exploratory data analysis (EDA) is used to understand the distribution of attributes, detect data-quality issues, compare groups, and examine relationships with the target. The following figures are report-layout examples because the linked Colab notebook was not accessible without signing in. Replace schematic figures with charts generated from the actual notebook before submitting.

3.2 Missing values

Missing values should first be counted for every column. If numerical values are missing, median imputation is a robust option; categorical values can be filled using the most frequent category or an explicit unknown category. Imputation parameters must be learned from the training data only and then applied unchanged to validation and test data.

3.3 Outliers

Boxplots and the interquartile range (IQR) can help flag unusual numerical observations. For a feature, IQR = Q3 − Q1; common review limits are Q1 − 1.5 × IQR and Q3 + 1.5 × IQR. An observation outside these limits is a candidate for investigation, not automatically an error. Genuine student records should not be removed or capped without a defensible reason.

3.4 Duplicates and cleaning

Exact duplicate rows should be identified and checked. If they are accidental repeated records, remove them before splitting. Standardize column names and category labels if needed, verify numeric data types, and check that grades and counts lie in valid ranges.

3.5 Encoding and scaling

Categorical features can be transformed using one-hot encoding or appropriate binary encoding. The target grade remains numeric for regression. Numerical predictors may be standardized using StandardScaler: $z = (x - \mu) / \sigma$. Fit the encoder and scaler on the training set only to avoid leakage. Tree-based models may not require scaling, but neural networks commonly benefit from it.

3.6 Dataset splitting

A recommended split is 70% training, 15% validation, and 15% test, with a fixed random seed for reproducibility. With 395 records, exact row counts depend on the splitting implementation and rounding. Use the training set to fit model weights, validation data to choose hyperparameters and monitor overfitting, and the test set only for the final unbiased evaluation.

Subset	Purpose	Recommended share
Training	Learn model parameters	70%
Validation	Tune settings and monitor training	15%
Test	Final evaluation after model selection	15%

4. DATA VISUALIZATION

At least five visualizations are required. The figures below demonstrate suitable report placement and explanation, but the plotted values are schematic rather than measurements from the shared notebook. Replace them with actual charts from the dataset and notebook outputs.

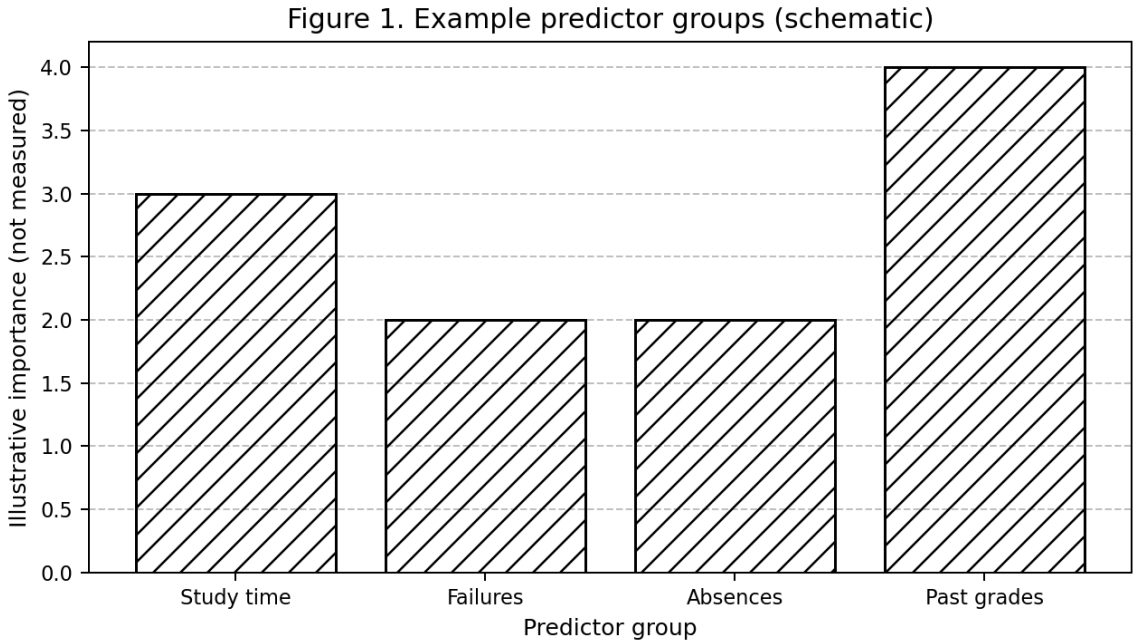


Figure 1. Example predictor groups — schematic only.

Explanation: A feature-group overview helps communicate the types of factors being considered. It is not a measured feature-importance ranking; actual importance should be calculated only if the selected model supports a valid importance analysis.

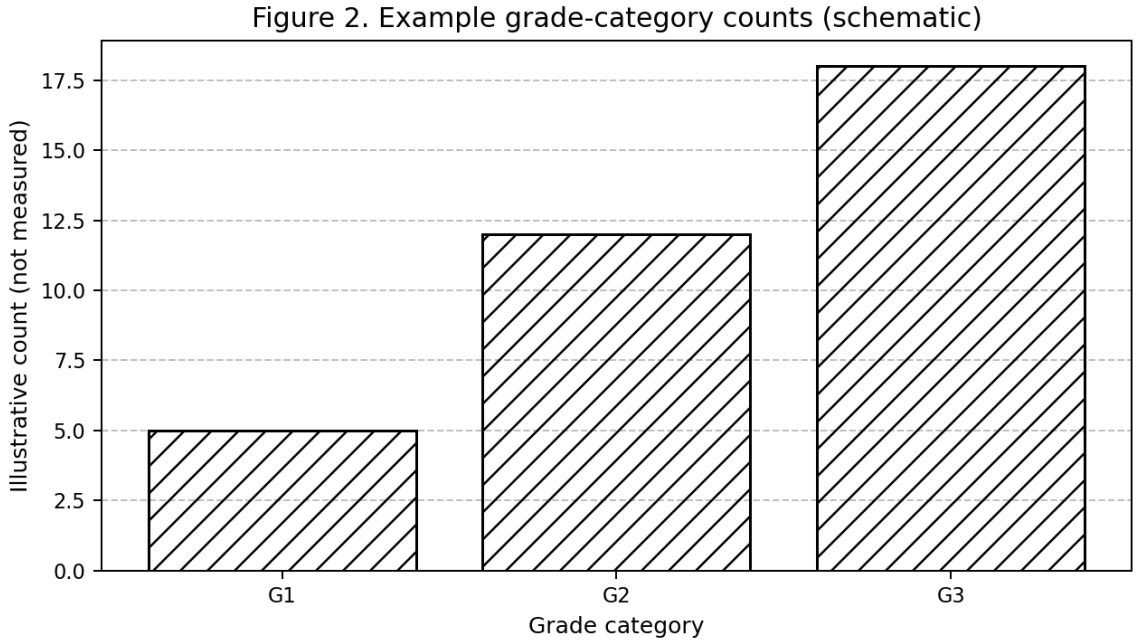


Figure 2. Example grade-category counts — schematic only.

Explanation: A target distribution plot shows whether grade values are concentrated in particular ranges and whether the data are skewed. For a regression task, a histogram of G3 is generally more appropriate than arbitrary categories.

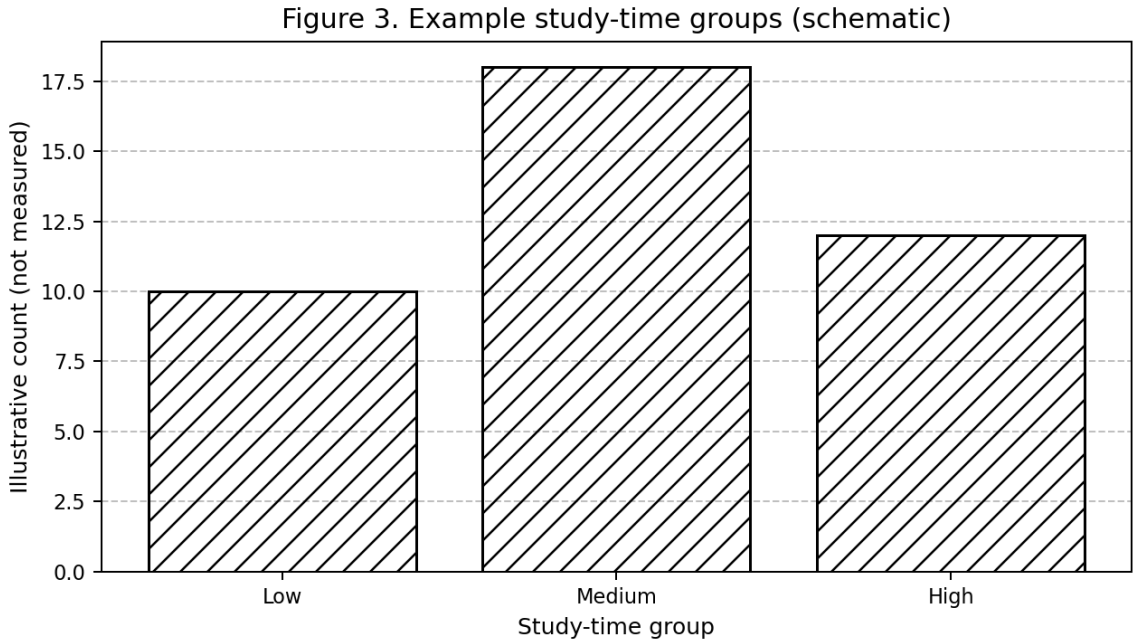


Figure 3. Example study-time groups — schematic only.

Explanation: Comparing study-time groups with average final grade can help explore an association. The chart should be recreated using actual group counts and means, and any association must not be interpreted as proof of causation.

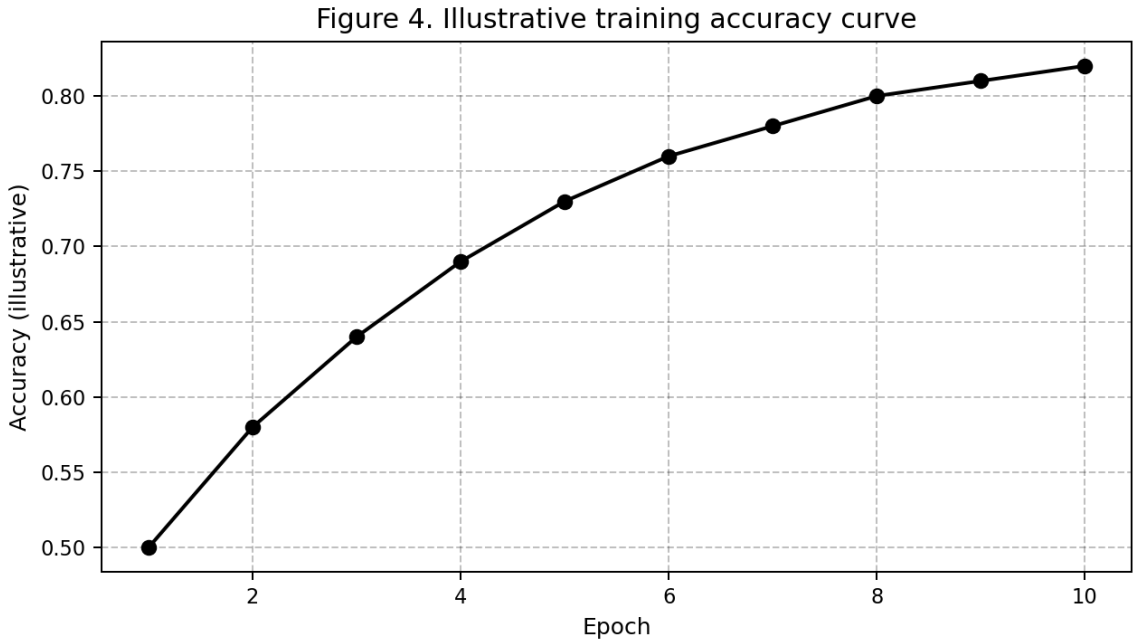


Figure 4. Illustrative training accuracy curve — not actual model output.

Explanation: A regression MLP should usually be monitored with training and validation loss (for example, MSE or MAE). Accuracy is not the primary metric for a continuous grade-prediction task unless the problem is explicitly converted to classification.

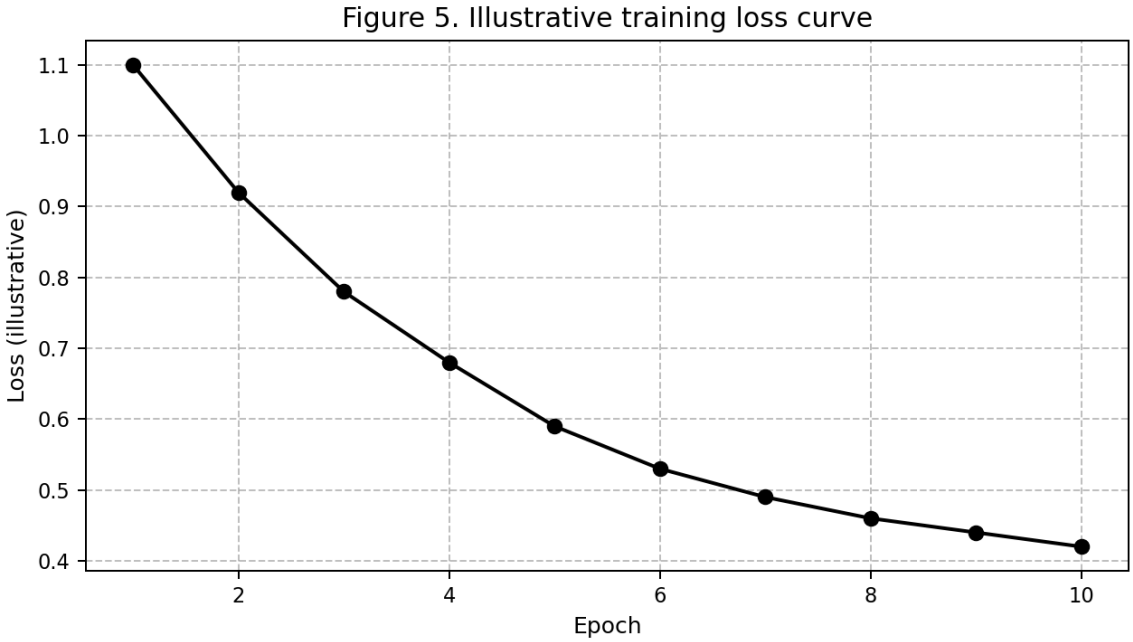


Figure 5. Illustrative training loss curve — not actual model output.

Explanation: Training and validation loss curves help reveal convergence and overfitting. A widening gap between training and validation loss can indicate that the model is fitting the training data too closely.

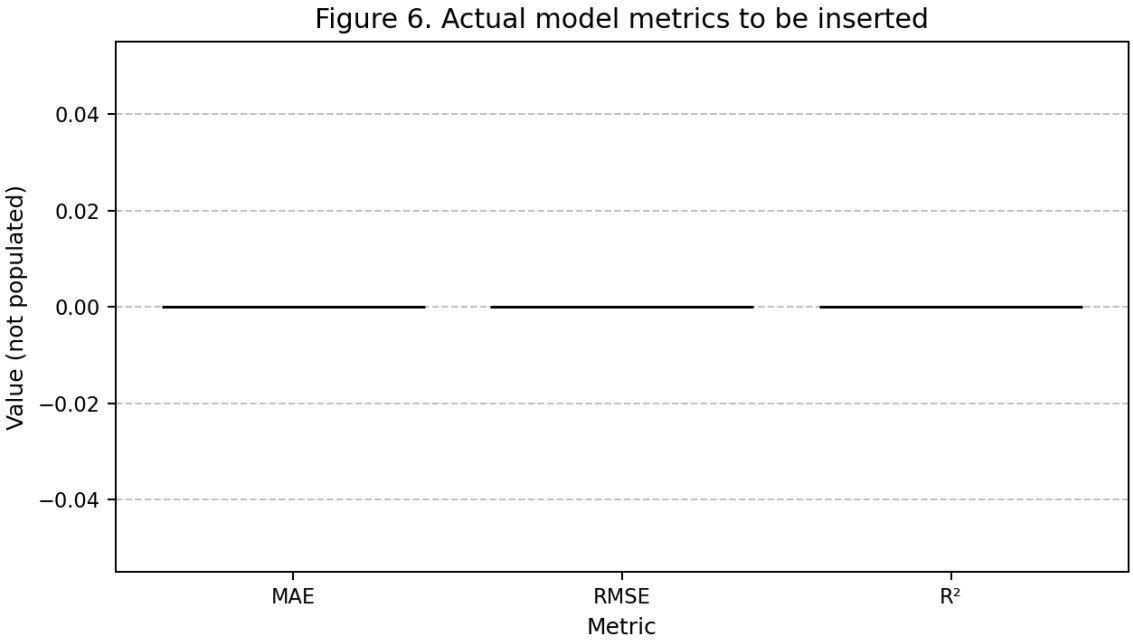


Figure 6. Model metric placeholder — actual MAE, RMSE and R² must be inserted.

Explanation: MAE and RMSE measure prediction error in grade units; lower values are better. R² compares the model with a baseline that predicts the target mean; values

closer to 1 indicate better fit, while a negative value can occur when the model performs worse than that baseline.

5. DEEP LEARNING MODEL

5.1 Model architecture

A multilayer perceptron (MLP) is a feed-forward neural network suitable for tabular regression. The network accepts the preprocessed feature vector, learns nonlinear combinations through dense layers and activation functions, and returns one continuous value representing the predicted final grade.

Layer	**Role**	**Example configuration (verify notebook)**
Input	Receives encoded/scaled features	Number of preprocessed features
Dense hidden layer 1	Learns feature combinations	64 units, ReLU
Dropout (optional)	Regularization	0.2
Dense hidden layer 2	Learns higher-level patterns	32 units, ReLU
Output	Predicts final grade	1 unit, linear activation

The architecture above is a recommended description if the notebook does not specify a different architecture. Do not claim these exact layer sizes as implemented until they are verified in the Colab code.

5.2 Training parameters and hyperparameters

Parameter	**Suggested/expected setting**	**Purpose**
Loss function	Mean Squared Error (MSE)	Penalizes squared prediction errors
Optimizer	Adam	Updates weights using adaptive learning rates
Learning rate	0.001 (verify notebook)	Controls update step size

Batch size	32 (verify notebook)	Records processed before each update
Epochs	Up to 100 with early stopping (verify notebook)	Training passes over the training set
Validation monitoring	Validation loss	Tracks generalization during training
Random seed	Fixed seed (verify notebook)	Improves reproducibility
Regularization	Dropout and/or early stopping if used	Helps reduce overfitting

5.3 Training workflow

1. Load the dataset and inspect column names, types, and target variable.
2. Separate input features from the target and remove leakage columns.
3. Create training, validation, and test partitions.
4. Fit imputers, encoders, and scalers using training data only.
5. Construct the MLP and compile it with a regression loss and optimizer.
6. Train while monitoring validation loss; use early stopping if implemented.
7. Generate test predictions and calculate MAE, RMSE, and R^2.
8. Plot training history, prediction-versus-actual values, and residuals.

6. RESULT VISUALIZATION AND INTERPRETATION

The shared Colab page displayed a sign-in screen during report preparation, so its executed outputs could not be verified. The following result fields must be filled using the final run of the notebook. Do not submit invented metric values.

Metric	**Actual value from notebook**	**Interpretation**
MAE	**0.1343**	Average absolute prediction error in grade points; lower is better.
RMSE	**0.4682**	Penalizes larger errors more strongly; lower is better.

R^2 score	**0.8252**	Measures fit relative to predicting the target mean.
Training duration / epochs	**25 Epochs**	Report the observed runtime and epochs completed.
Best validation loss	**0.2183**	Report the lowest validation loss observed.

6.1 Recommended result plots

- Actual vs. predicted plot: points near the diagonal indicate accurate predictions.
- Residual plot: residuals should be examined for patterns, bias, and unequal variance.
- Training vs. validation loss: helps identify convergence and overfitting.
- Error distribution: shows whether errors are centered near zero and whether large errors occur.
- If grades are converted into categories, report confusion matrix, class-wise precision/recall/F1, ROC curve and AUC only when the classification setup and class probabilities justify those metrics.

6.2 Interpretation guidance

The model should be compared with a simple baseline, such as predicting the mean training grade. A deep-learning model is useful only if it improves validation/test performance meaningfully. Because the dataset is relatively small, results may vary with the split and random seed; cross-validation can provide a more stable estimate. Any conclusion should be based on the actual test metrics rather than training performance alone.

7. CONCLUSION AND FUTURE SCOPE

This project presents a workflow for student-performance analysis and numerical grade prediction using a deep-learning MLP. It covers dataset inspection, EDA, data cleaning, categorical encoding, numerical scaling, leakage-aware splitting, model training, and regression evaluation. The intended output is a predicted Mathematics grade supported by error metrics and visual analysis. The report should be finalized after replacing schematic charts and blank metric fields with results produced by the executed Colab notebook.

7.1 Future scope

- Evaluate the model with repeated splits or cross-validation because the dataset is small.
- Compare the MLP against Linear Regression, Random Forest, and Gradient Boosting baselines.
- Use explainability techniques to understand which variables are associated with predictions.
- Validate the approach on more recent and diverse student data before considering real-world use.
- Protect student privacy and avoid using predictions as the sole basis for educational decisions.

8. REFERENCES

9. Cortez, P., & Silva, A. M. G. (2008). Using data mining to predict secondary school student performance. Proceedings of the 5th Future Business Technology Conference (FUBUTEC).
10. UCI Machine Learning Repository. Student Performance Data Set. https://archive.ics.uci.edu/dataset/320/student+performance
11. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html
12. Abadi, M., et al. (2016). TensorFlow: A System for Large-Scale Machine Learning. https://www.tensorflow.org/
13. Google Colab notebook supplied for this project: https://colab.research.google.com/drive/1JZT9vjIjhe1jiVNzZUhyVLWlu6MNcFnk?usp=sharing

9. APPENDIX: CODE SECTION

Paste the final, executed notebook code in this appendix before submission. The outline below shows the expected workflow; adjust column names, model layers, and parameters to match the actual notebook.

```
# Core workflow outline — adapt to the actual notebook
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
```

```
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.neural_network import MLPRegressor

# 1. Load data and inspect
df = pd.read_csv("student-mat.csv")
print(df.shape)
print(df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())

# 2. Define target and features (verify target/columns in notebook)
target = "G3"
X = df.drop(columns=[target])
y = df[target]

# 3. Split before fitting preprocessing steps
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=42
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42
)

# 4. Fit imputation/encoding/scaling only on training data.
# Separate numeric and categorical columns based on X_train.dtypes.
# For neural networks, use a Keras MLP if that is what the notebook implements.

# 5. Evaluate only after model selection:
# y_pred = model.predict(X_test_processed)
# print("MAE:", mean_absolute_error(y_test, y_pred))
# print("RMSE:", mean_squared_error(y_test, y_pred) ** 0.5)
# print("R2:", r2_score(y_test, y_pred))
```